

Section (Algorithms)

Eng. Abdulrahman
ElMadkhoun

Standard Base Class & Utilities

```
# Shared helper functions used across the scripts
INITIAL_STATE = (
    (2, 8, 3),
    (1, 6, 4),
    (7, 0, 5)
)

GOAL_STATE = (
    (1, 2, 3),
    (8, 0, 4),
    (7, 6, 5)
)

# Moves defined strictly in order: Left, Up, Right, Down
# Format: (row_offset, col_offset, move_name)
MOVES = [
    (0, -1, "Left"),
    (-1, 0, "Up"),
    (0, 1, "Right"),
    (1, 0, "Down")
]
```

Standard Base Class & Utilities

```
class Node:
    def __init__(self, state, parent=None, action=None, depth=0, cost=0):
        self.state = state          # The 3x3 board tuple configuration
        self.parent = parent        # Reference to parent node (used for path reconstruction)
        self.action = action        # Move taken to reach this node ("Left", "Up", etc.)
        self.depth = depth          # Tree depth (g(n) path cost)
        self.cost = cost            # Accumulated cost field (used in A* / heuristic algorithms)
        self.children = []          # List of references to generated child nodes (for visual tree generation)

    def get_blank_pos(self):
        for r in range(3):
            for c in range(3):
                if self.state[r][c] == 0:
                    return r, c
```

Standard Base Class & Utilities

```
def generate_successors(self):
    r, c = self.get_blank_pos()
    successors = []
    for dr, dc, move_name in MOVES:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            # Move tile into blank space
            new_state = [list(row) for row in self.state] #Converts immutable tuple state to
            new_state[r][c], new_state[nr][nc] = new_state[nr][nc], new_state[r][c]
            tuple_state = tuple(tuple(row) for row in new_state)
            child = Node(tuple_state, parent=self, action=move_name, depth=self.depth + 1)
            successors.append(child)
    return successors
```

BFS

```
from collections import deque
# Imports 'deque' (double-ended queue), which allows fast O(1) appends and pops from both ends.

def bfs(initial_state, goal_state):
    root = Node(initial_state)
    # Wraps the raw tuple board into a Node object at depth 0.

    if initial_state == goal_state:
        return root, root
        # Edge case: Checks if the starting state is already the goal state.

    queue = deque([root])
    # Initializes the FIFO queue with the root node.

    visited = {initial_state}
    # Initializes a set with the root state to track visited configurations and avoid infinite loops.
```

BFS

```
while queue:
    # Continues processing nodes as long as there are unexpanded nodes in the queue.

    current = queue.popleft()
    # Removes and returns the leftmost (oldest) node in the queue (FIFO behavior).

    for child in current.generate_successors():
        # Generates all valid child states by attempting moves in the order: Left, Up, Right, Down.

        if child.state not in visited:
            # Ensures we do not process states we have already encountered.

            visited.add(child.state)
            # Marks the new state as visited immediately upon generation.

            current.children.append(child)
            # Links the child to the parent node to construct the visual search tree.

            if child.state == goal_state:
                return root, child
                # Early Goal Test: Returns the full tree and goal node as soon as the goal is generated.

            queue.append(child)
            # Adds the child to the end of the queue for later exploration.
```

DFS

```
def dfs(initial_state, goal_state, max_depth=15):  
    root = Node(initial_state)  
    # Creates the starting root node.  
  
    stack = [root]  
    # Initializes the stack using a Python list with the root node.  
  
    visited = {initial_state}  
    # Set to record visited configurations.  
  
    while stack:  
        # Runs until there are no remaining nodes in the stack.  
  
        current = stack.pop()  
        # Pops and returns the top (most recently added) element from the stack (LIFO behavior).  
  
        if current.state == goal_state:  
            return root, current  
            # Late Goal Test: Checks if the node popped from the stack is the goal state.
```

DFS

```
if current.depth < max_depth:
    # Prunes search paths that exceed the maximum defined depth limit.

    for child in reversed(current.generate_successors()):
        # Reverses successors so that when pushed to the stack, Left is popped first (maintaining

        if child.state not in visited:
            # Filters out previously visited states.

            visited.add(child.state)
            # Marks the state as visited.

            current.children.append(child)
            # Links the child to the current node for tree visualization.

            stack.append(child)
            # Pushes the child node onto the top of the stack.

return root, None
# Returns the tree and None if no path reaches the goal within the depth boundary.
```


Best First Manhattan Distance

```
import heapq

def manhattan_distance(state, goal_state):
    goal_pos = {}
    # Dictionary mapping each tile number to its exact target (row, col) coordinates.

    for r in range(3):
        for c in range(3):
            goal_pos[goal_state[r][c]] = (r, c)
            # Populates goal coordinate mappings.

    dist = 0
    # Initializes distance sum accumulator.

    for r in range(3):
        for c in range(3):
            val = state[r][c]
            if val != 0:
                # Calculates displacement distance for non-blank tiles only.

                gr, gc = goal_pos[val]
                dist += abs(r - gr) + abs(c - gc)
                # Sums grid distance steps required for tile to reach goal location.

    return dist
```

Best First Manhattan Distance

```
def best_first_manhattan(initial_state, goal_state):
    root = Node(initial_state)
    counter = 0
    pq = [(manhattan_distance(initial_state, goal_state), counter, root)]
    # Priority Queue ordered by Manhattan Distance h(n).

    visited = {initial_state}

    while pq:
        h, _, current = heapq.heappop(pq)
        # Pops node with lowest Manhattan distance h(n).

        if current.state == goal_state:
            return root, current

        for child in current.generate_successors():
            if child.state not in visited:
                visited.add(child.state)
                current.children.append(child)
                counter += 1
                heapq.heappush(pq, (manhattan_distance(child.state, goal_state), counter, child))
                # Pushes generated child node prioritized by manhattan_distance heuristic.

    return root, None
```

A* Misplaced Tiles

```
import heapq

def misplaced_tiles(state, goal_state):
    return sum(
        1 for r in range(3) for c in range(3)
        if state[r][c] != 0 and state[r][c] != goal_state[r][c]
    )

def a_star_misplaced(initial_state, goal_state):
    root = Node(initial_state, cost=0)
    # Sets starting cost  $g(n) = 0$ .

    counter = 0
    h_root = misplaced_tiles(initial_state, goal_state)

    pq = [(root.cost + h_root, counter, root)]
    # Priority Queue stores tuple prioritized by total cost:  $f(n) = g(n) + h(n)$ .

    visited = {initial_state: 0}
    # Dictionary storing state -> lowest  $g(n)$  cost found so far.
```

A* Misplaced Tiles

```
while pq:
    f, _, current = heapq.heappop(pq)
    # Retrieves node with lowest f(n) total estimated path cost.

    if current.state == goal_state:
        return root, current
        # Path optimality guarantee met when goal is popped from queue.

    for child in current.generate_successors():
        child.cost = current.depth
        # Sets actual path cost g(n) equal to tree depth level.

        if child.state not in visited or child.cost < visited[child.state]:
            # Checks if state is unvisited or reached via a lower-cost path.

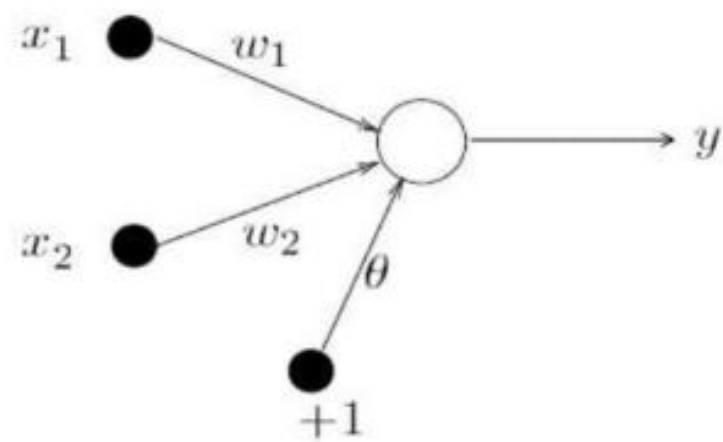
            visited[child.state] = child.cost
            # Updates dictionary with improved g(n) path cost.

            current.children.append(child)
            # Adds child to search tree display model.

            counter += 1
            f_cost = child.cost + misplaced_tiles(child.state, goal_state)
            # Computes evaluation function value: f(n) = g(n) + h(n).

            heapq.heappush(pq, (f_cost, counter, child))
            # Pushes child node onto priority queue sorted by total f_cost.
```

Simple Perceptron



Perceptron Learning

$$Y = \mathcal{F}(W_i * X_i + W_j * X_j + W_k * X_k + \dots)$$

$$\text{Weight Update: } W_i = W_i + \eta d(n) X_i$$

(n)

Learning
rate

[Target class – current
output]

Input

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0, \\ 0 & \text{otherwise} \end{cases}$$

$$\text{sgn}(s) = \begin{cases} 1 & \text{if } s > 0 \\ -1 & \text{otherwise.} \end{cases}$$



```
import numpy as np

# Activation function
def activation(z):
    return 1 if z > 0 else 0

# Perceptron training with early stopping
def perceptron_train(X, y, eta=0.2, epochs=100, initial_weights=None):
    # Initialize weights
    if initial_weights is None:
        w = np.zeros(3)
    else:
        w = np.array(initial_weights, dtype=float)
```

```
for epoch in range(epochs):
    errors = 0
    print(f"\nEpoch {epoch+1}")
    for i in range(len(X)):
        x = X[i]
        target = y[i]
        z = np.dot(w, x)
        pred = activation(z)
        update = eta * (target - pred)
        w = w + update * x
        if update != 0:
            errors += 1
        print(f"p{i+1}: x={x[1:]}, target={target}, pred={pred}, update={update:.3f}, w={w}")

    if errors == 0:
        print(f"\nConverged after {epoch+1} epochs.")
        return w

print("\nFinished all epochs.")
return w
```



```
# Dataset
X = np.array([
    [1, 1, 1],      # p1
    [1, 2, -2],     # p2
    [1, -1, -1.5],  # p3
    [1, -2, -1],    # p4
    [1, -2, 1],     # p5
    [1, 1.5, -0.5]  # p6
])
y = np.array([1, 0, 0, 0, 1, 1])

# Initial weights: w0=0, w1=1, w2=0.5
initial_weights = [0, 1, 0.5]

# Train perceptron
final_weights = perceptron_train(X, y, eta=0.2, epochs=100, initial_weights=initial_weights)
print("\nFinal weights (w0, w1, w2):", final_weights)
```

...

Epoch 1

p1: x=[1. 1.], target=1, pred=1, update=0.000, w=[0. 1. 0.5]
p2: x=[2. -2.], target=0, pred=1, update=-0.200, w=[-0.2 0.6 0.9]
p3: x=[-1. -1.5], target=0, pred=0, update=0.000, w=[-0.2 0.6 0.9]
p4: x=[-2. -1.], target=0, pred=0, update=0.000, w=[-0.2 0.6 0.9]
p5: x=[-2. 1.], target=1, pred=0, update=0.200, w=[0. 0.2 1.1]
p6: x=[1.5 -0.5], target=1, pred=0, update=0.200, w=[0.2 0.5 1.]

Epoch 2

p1: x=[1. 1.], target=1, pred=1, update=0.000, w=[0.2 0.5 1.]
p2: x=[2. -2.], target=0, pred=0, update=0.000, w=[0.2 0.5 1.]
p3: x=[-1. -1.5], target=0, pred=0, update=0.000, w=[0.2 0.5 1.]
p4: x=[-2. -1.], target=0, pred=0, update=0.000, w=[0.2 0.5 1.]
p5: x=[-2. 1.], target=1, pred=1, update=0.000, w=[0.2 0.5 1.]
p6: x=[1.5 -0.5], target=1, pred=1, update=0.000, w=[0.2 0.5 1.]

Converged after 2 epochs.

Final weights (w0, w1, w2): [0.2 0.5 1.]

KMeans

```
def plot_clusters(data, clusters, centroids, iteration):
    data = np.array(data)
    K = len(centroids)
    colors = ['red', 'blue', 'green', 'purple', 'orange', 'brown']

    plt.figure(figsize=(6,6))

    # Plot points by cluster
    for k in range(K):
        cluster_points = data[clusters == k]
        if len(cluster_points) > 0:
            plt.scatter(cluster_points[:,0], cluster_points[:,1],
                        s=80, c=colors[k % len(colors)], label=f"Cluster {k+1}")

    # Plot centroids
    for k, c in enumerate(centroids):
        plt.scatter(c[0], c[1], s=200, c='black', marker='x')
        plt.text(c[0]+0.1, c[1]+0.1, f"C{k+1}", fontsize=12)

    plt.title(f"K-Means Clustering (Iteration {iteration})")
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.legend()
    plt.grid(True)
    plt.show()
```

KMeans

```
def kmeans(data, K, max_iters=100):
    data = np.array(data, dtype=float)
    centroids = data[:K].copy() # Initialize centroids using the first K points

    for iteration in range(1, max_iters+1):
        print(f"\nIteration {iteration}")

        # ----- Assign clusters -----
        clusters = np.zeros(len(data), dtype=int)
        for i, point in enumerate(data):
            min_dist = float('inf')
            for k, centroid in enumerate(centroids):
                # Compute Euclidean distance
                dist = np.sqrt(np.sum((point - centroid)**2))
                if dist < min_dist:
                    min_dist = dist
                    clusters[i] = k
```

KMeans

```
# Print points in each cluster
for k in range(K):
    print(f"Cluster {k+1}: {data[clusters == k]}")

# ----- Plot clusters -----
plot_clusters(data, clusters, centroids, iteration)

# ----- Recompute centroids -----
new_centroids = []
for k in range(K):
    if np.any(clusters == k):
        new_centroids.append(data[clusters == k].mean(axis=0))
    else:
        new_centroids.append(centroids[k])
new_centroids = np.array(new_centroids)

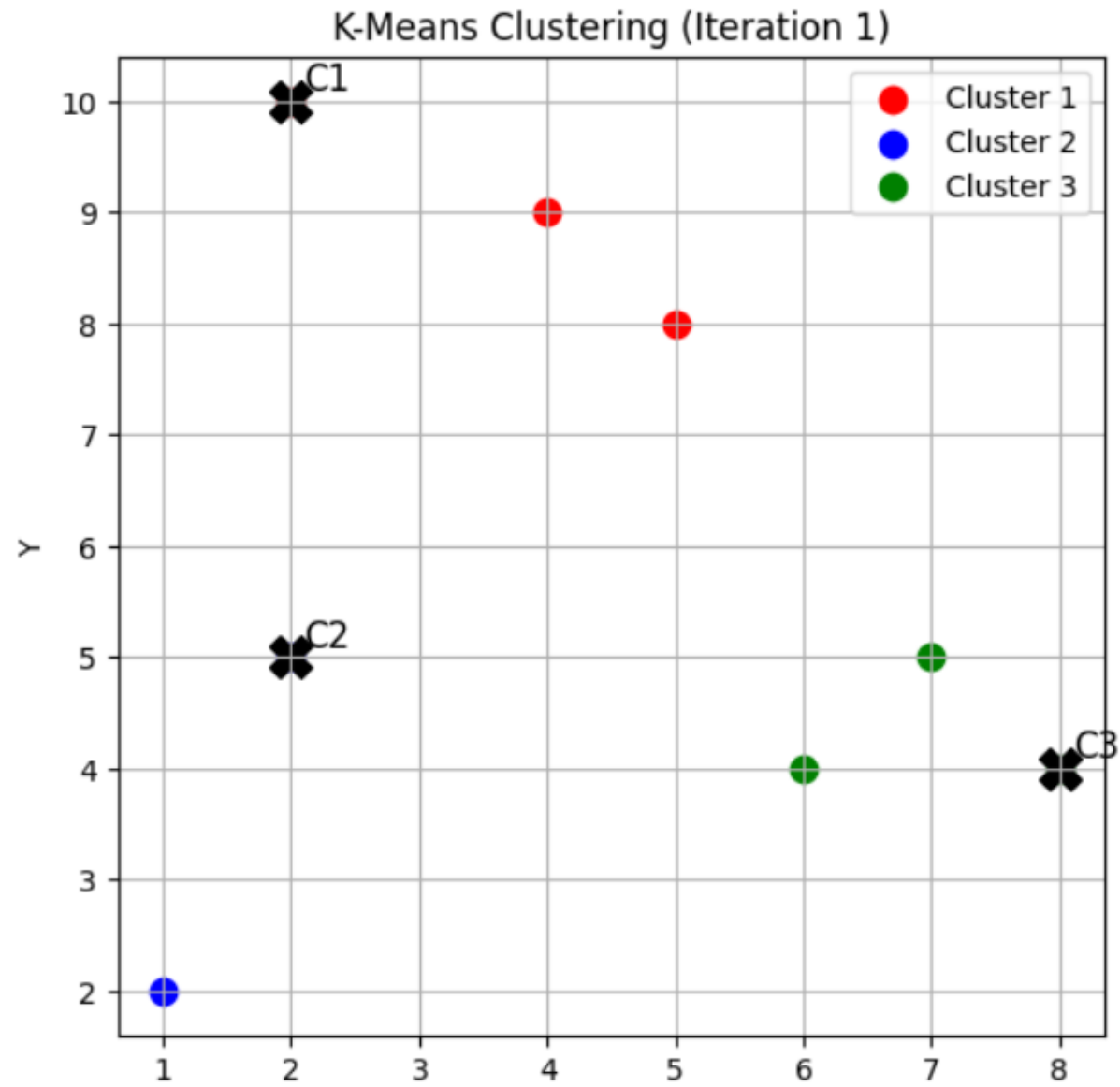
print("New centroids:", new_centroids)

# ----- Convergence check -----
if np.allclose(new_centroids, centroids):
    print("\nConverged.")
    break

centroids = new_centroids

return clusters, centroids
```

KMeans



KMeans

